Session 6 · Deck 6.B1

Krusell–Smith: When the Distribution Is a State

Zhigang Feng

Thursday, October 29, 2026

© Zhigang Feng • Redistribution prohibited without permission

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 6.B1 ▼

S1 ✓	S2 ✓	S3 ✓	S4 ✓	S5 ✓	S6	S7	S8	S9	S10
Sep 24	Oct 1	Oct 8	Oct 15	Oct 22	Oct 29	Nov 5	Nov 12	Nov 19	Dec 3
Intro	HA · OLG	Num. DP	Numerics	Numerics	HA classic	ML	RL	Text	Agents
Growth	Policy	Python	I	II	The wall	Deep nets	HA + DL	LLMs	Projects
trunk	branches	the machine · classical methods			the wall	new tools		new data	colleagues

Build it on paper → solve it on the machine → push it to the frontier with AI

Block A — Aiyagari: 6.A1 the household, from Bellman to Euler · 6.A2 the population and the price.
Block B: 6.B1 Krusell–Smith · 6.B2 transitions and the life cycle.

© Zhigeng Fang
Redistribution prohibited without permission

Roadmap: Five Parts

2.B1 defined the equilibrium and stopped at "it closes". This deck computes it, by the two routes of 6.A1, and grades the answer.

Part	Topic	The question it answers
1	Two kinds of risk	Why must households forecast the distribution, and what is the bet?
2	The household with two more states	The inner problem, given the rule: Bellman on (a, ε, K, z), by VFI or by EGM.
3	The loop	The outer problem: simulate for the data on K, regress for the new rule, damp.
4	Is it right?	"Converged"; what R^2 and Den Haan's test each measure.
5	Why it works, where it stops	Why one moment suffices; what the whole distribution costs.

© Zhigang Feng Redistribution prohibited without permission.

Idea

Krusell and Smith's bet: forecast the distribution by its mean. The household then has four states — 6.A1's methods still apply — and the bet is a fixed point in four numbers.

The Economy: Two Kinds of Risk

The economy every comparison in the literature uses: Krusell and Smith's, as parameterized by Den Haan, Judd and Juillard (2010). Quarterly.

Aggregate: productivity $z \in \{0.99,\ 1.01\}$, recession or boom, each lasting 8 quarters on average.

Individual: employed or unemployed. Unemployment is 10% in a recession and 4% in a boom; a spell lasts 2.5 quarters in a recession, 1.5 in a boom. The unemployed receive a benefit of 15% of the wage, financed by a flat tax on the employed within the quarter.

Preferences and technology: log utility, $\beta = 0.99$; $Y = zK^{\alpha}L^{1-\alpha}$, $\alpha = 0.36$, $\delta = 0.025$; $a' \geq 0$. Prices depend on (K, z): $r = \alpha z(K/L)^{\alpha-1}$, $w = (1-\alpha)z(K/L)^{\alpha}$.

A transfer, not 6.B2's tax. The benefit is part of the published calibration, paid within the quarter, nothing kept; it is absent from 2.B1 and Block A, and $\mu = 0$ removes it.

Example: the joint chain, built and checked

Over (recession, boom) × (unemployed, employed), *built* from the durations plus Krusell and Smith's two restrictions on job finding when z switches:

0.525	0.350	0.031	0.094
0.039	0.836	0.002	0.123
0.094	0.031	0.292	0.583
0.009	0.116	0.024	0.851

Den Haan, Judd & Juillard, *JEDC* 34 (2010), Tables 1–2 (the matrix matches theirs to 4×10^{-7}, asserted before running); Krusell & Smith, *JPE* 106 (1998).

Ledger `ks.calibration`, `ks.transition_matrix`.

© Yiming Feng (distribution prohibited without permission)

Tomorrow's interest rate depends on tomorrow's capital, the mean of tomorrow's distribution: the distribution is a state.

$$V(a,\varepsilon,z,\Gamma) = \max_{0\le a'\le \mathrm{coh}} \Big\{ u(\mathrm{coh}-a') + \beta\,\mathbb{E}\big[V(a',\varepsilon',z',\Gamma') \,\big|\, \varepsilon, z\big] \Big\}$$

Here $\mathrm{coh} = \big(1 + r(K,z) - \delta\big)a + \mathrm{income}(\varepsilon, K, z)$, $K = \int a\, d\Gamma$ and $\Gamma' = H(\Gamma, z, z')$. The household needs r and w tomorrow, so it needs K', so it needs Γ' — the whole distribution, carried forward by the law of motion H (2.B1). No grid can hold Γ as a state variable.

Notation. Γ is the distribution as a state; λ (6.A2) was the invariant one; H is the law of motion households believe.

Idea

In Block A the distribution sat still and prices were numbers. With an aggregate shock that keeps coming, prices are functions of the distribution.

2.B1, "Why the Household Cannot Avoid Forecasting Γ", and "Common Noise Is Why Γ Must Be a State".

© Zhaonan Feng Reproduction prohibited without permission

2.B1's equilibrium with aggregate risk. New since Block A: prices depend on (K, z), and the forecast H is part of the equilibrium.

Definition

A **recursive competitive equilibrium** is a law of motion H, a value V and a policy $a'(a, \varepsilon, z, \Gamma)$, prices $r(K, z)$, $w(K, z)$ and a tax $\tau(z)$ such that:

1. **Households:** given the prices, τ and H, V solves the Bellman equation of the last frame and a' is its policy.
2. **Firm:** $r = \alpha z (K/L_z)^{\alpha-1}$, $w = (1-\alpha) z (K/L_z)^{\alpha}$, with $K = \int a \, d\Gamma$ and L_z the labour employed in state z.
3. **Transfer:** a flat tax on the employed pays the benefit, $\mu = 15\%$ of w, within the period: $\tau(z)\, \bar{l}\, (1 - u_z) = \mu\, u_z$. Nothing is kept.
4. **Consistency:** H is what the policy produces: $\Gamma'(\mathcal{B}, \varepsilon') = \sum_{\varepsilon} \pi(\varepsilon' \mid \varepsilon, z, z')\, \Gamma\big(\{a : a'(a, \varepsilon, z, \Gamma) \in \mathcal{B}\}, \varepsilon\big)$ for every set $\mathcal{B}$.

Idea

Condition 4 is new: beliefs must match what the choices produce.

What we compute

An *approximate* equilibrium: condition 4 on a log-linear rule for K (next frame).

© Zhigang Feng · Redistribution prohibited without permission

2.B1, "Recursive Competitive Equilibrium with Aggregate Risk"; Krusell & Smith, *JPE* 106 (1998).

Krusell and Smith's Bet: Forecast With the Mean

Replace Γ by its first moment, guess a log-linear law for it, and require the guess to reproduce itself.

$$\log K' = \kappa_0(z) + \kappa_1(z)\,\log K.$$

Four numbers, one line for each aggregate state. The household's state shrinks from $(a, \varepsilon, z, \Gamma)$ to (a, ε, K, z).

The equilibrium requirement. The rule households *use* must be the rule their behaviour *generates*: simulate the economy with it, regress $\log K'$ on $\log K$, and get the same four numbers back. A fixed point in κ.

Example: the converged rule

recession: $\log K' = 0.1240 + 0.9655\,\log K$
boom: $\log K' = 0.1371 + 0.9634\,\log K$

Left alone, the rule settles at $K = 36.4$ in a long recession and 42.4 in a long boom.

Pitfall

The bet is checked, not assumed: Part 4 grades the rule, Part 5 says when it fails.

© Zhigang Feng • Redistribution prohibited without permission

Ledger `ks.solution` (`kappa_bad`, `kappa_good`), `ks.diagnostics.K` (`rule_rest_bad`, `rule_rest_good`).

The Function Now Has Four Arguments

Given the rule, the aggregate state enters only through prices today and the forecast K' tomorrow. It is 6.A1's problem with a table of prices.

$$V(a,s,K) = \max_{a'} \Big\{ u\big(\mathrm{coh} - a'\big) + \beta \sum_{s'} P_{ss'}\, V\big(a', s', K'(K,z)\big) \Big\},$$

with $s = (z, \varepsilon)$ one of four states, $\mathrm{coh} = R(K,z)\,a + \mathrm{inc}(\varepsilon, K, z)$ where $\mathrm{inc} = (1-\tau(z))\,\bar{l}\,w(K,z)$ if employed and $\mu\, w(K,z)$ if not, and $K'(K,z) = \exp\big(\kappa_0(z) + \kappa_1(z)\log K\big)$.

The grid. 120 asset nodes on $[0, 300]$, dense near the limit; 7 nodes for K on $[30, 50]$, equally spaced in $\log K$; the four values of s. In all $120 \times 4 \times 7 = 3{,}360$ states.

Idea

The joint chain P is 4×4, so the expectation is one matrix product, as in 6.A1. What is new is that K' falls *between* the K-nodes: V must be read there.

Why $\log K$

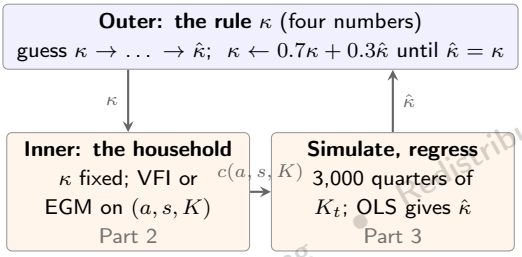
The rule is linear in $\log K$, so nodes equally spaced in $\log K$ put the forecast the same relative distance from its neighbours everywhere.

© Zhigang Feng
Redistribution prohibited without permission

Ledger `ks.household_methods.state_count`, `ks.settings`.

2 · A Nested Fixed Point: the Household Given κ, the Loop on κ

Two problems, one inside the other. The inner one is 6.A1's household with the rule κ held fixed; the outer one changes κ and runs the inner one again.



Keep them apart: the inner problem is solved to tolerance at the current κ and knows nothing of the loop; consistency is the outer loop's job alone.

Ledger `ks.solution.rows`; `solve_ks` **in** `s06_ks.py`.

Against Block A

6.A2's outer loop was on one number, r; here it is on four, around a household that takes the rule — hence tomorrow's prices — as given.

How often each runs

16 outer passes; inside them 7,360 household sweeps and $16 \times 3{,}000$ simulated quarters. The inner problem takes the time (next frame).

© Zhigang Feng · Redistribution prohibited without permission

2 · VFI for the Inner Problem: Interpolate in $\log K$, Then in a

The inner problem with κ fixed, by the value route Krusell and Smith took: 6.A1's interpolated VFI, with one more interpolation for tomorrow's K.

1: $m_0, \theta \leftarrow$ where $K'(K_m, z)$ falls between K-nodes ▷ linear in $\log K$
2: **repeat**
3: $EV(a_j, s, K_m) \leftarrow$ $\sum_{s'} P_{ss'} \left[(1-\theta) V(a_j, s', K_{m_0}) + \theta V(a_j, s', K_{m_0+1})\right]$
4: each (a_i, s, K_m): $a' \leftarrow \arg\max_x u(\text{coh} - x) + \beta\, EV(x, s, K_m)$ ▷ golden section
5: $V^{\text{new}} \leftarrow$ the maxima; $d \leftarrow \max |V^{\text{new}} - V|$; $V \leftarrow V^{\text{new}}$
6: Howard: 30 updates of V with a' held fixed ▷ 4.B3
7: **until** $d < 10^{-8}$

Or the Euler route: 6.A1's EGM with the same $\log K$ weights (tomorrow's c read at the forecast K'; the limit a comparison): 1,257 sweeps, 0.31 s against VFI's 2.3 s, the policies agreeing to 0.11% in c where households are. The loop runs it (How It Works 2).

Measured, converged rule

Plain VFI (10^{-6}): **1,382** sweeps, 40 s — 3.A2 predicts $\ln 10^{-6} / \ln 0.99 \approx 1{,}375$. With Howard: **62** maximizations, 2.3 s.

Pitfall

If K' leaves the K-grid, Line 3 would extrapolate in silence; the code stops and asks for a wider grid.

Krusell & Smith (1998) solved the household by value function iteration. Line 3: 4.B1, "Two Dimensions"; Line 4 reads EV linearly between asset nodes.

Ledger `ks.household_methods` **(**`vfi_plain, vfi_howard, egm, agreement`**).**

© Zhuoran Song · Redistribution prohibited without permission

The Loop: Solve, Simulate, Regress, Update

A fixed point in four numbers: the rule households use must equal the rule their behaviour implies.

1: $\kappa \leftarrow$ a guess
2: **repeat**
3: household: $c(a, s, K)$ given κ ▷ Part 2, warm-started
4: simulate: push the histogram 3,000 quarters along one z path; record K_t ▷ next frame
5: regress $\log K_{t+1}$ on $\log K_t$, separately for each z, after 500 quarters ▷ $\hat{\kappa}$: the frame after
6: **if** $\max|\hat{\kappa} - \kappa| < 10^{-5}$ **stop**
7: $\kappa \leftarrow 0.7\,\kappa + 0.3\,\hat{\kappa}$ ▷ damping, 4.A1
8: **until** stopped

`solve_ks` in `s06_ks.py`; **Lab 6, block B1**.

Why damp

A new rule changes behaviour, which changes the rule the regression finds. Taking the full step can overshoot; 30% of it does not (4.A1, "Take Only Part of the Step").

Example

One z path, drawn once with a fixed seed, is reused at every pass: the rule changes, the luck does not.

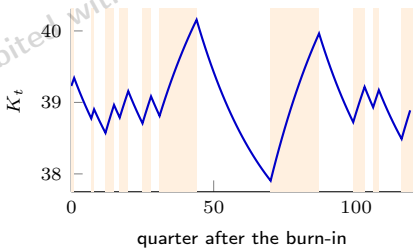
© Zhigang Feng
Redistribution prohibited without permission

3 · Simulate: Push the Histogram One Quarter at a Time, and Record K_t

The data for the regression are the series K_t. They come from pushing a distribution, not a panel: one quarter is 6.A2's lottery at the prices of that quarter.

1: $\lambda_0 \leftarrow$ a start with $K_0 = 40$; draw $z_0, \ldots, z_{3000}$ ▷ fixed seed
2: **for** $t = 0, \ldots, 2999$ **do**
3: $K_t \leftarrow \sum h\, \lambda_t(h, \varepsilon)$: **record it** ▷ the datum
4: prices at (K_t, z_t); stop if K_t leaves the K-grid ▷ guard
5: each node: $a' \leftarrow$ the policy read at K_t ▷ $\log K$, then a
6: lottery of a'; employment by P's (z_t, z_{t+1}) block ▷ 6.A2
7: **end for**
8: drop the first 500 quarters ▷ burn-in

Only the aggregate shocks are drawn: the series K_t has no sampling error.



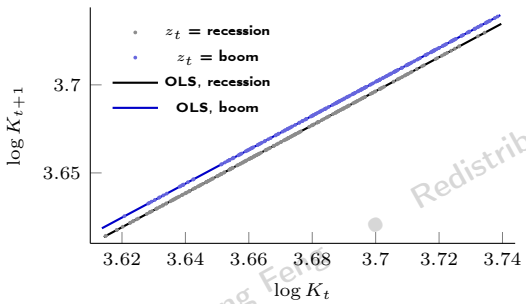
120 of the 2,500 quarters used; shaded: booms. K_t drifts up in booms and down in recessions — the two lines of the regression.

Ledger `pictures.ks_regression`; `ks_lottery_step` **and** `simulate` **in** `s06_ks.py`. **Fine grid: 3,001 nodes** 0.1 **apart.**

© Zhigang Feng · Redistribution prohibited without permission

Regress: the Series K_t Gives the New Rule

Two regressions, one per aggregate state: the quarters in a recession and the quarters in a boom, two coefficients each. Then the damped update.



For each z: ordinary least squares of $\log K_{t+1}$ on a constant and $\log K_t$ over the quarters with $z_t = z$, $\hat{\kappa}(z) = (X^\top X)^{-1} X^\top y$ — two numbers per state. One pooled line would run through two clouds at different heights: the intercept depends on z.

Measured, last pass

recession: $n = 1,127$, $\hat{\kappa} = (0.1239,\ 0.9655)$, $R^2 = 0.9999985$

boom: $n = 1,372$, $\hat{\kappa} = (0.1371,\ 0.9634)$, $R^2 = 0.9999991$

Update $\kappa \leftarrow 0.7\,\kappa + 0.3\,\hat{\kappa}$; stop when $\max|\hat{\kappa} - \kappa| < 10^{-5}$ — here 7×10^{-6}.

Ledger `pictures.ks_regression` **(equal to** `ks.solution.kappa_regression`**);** `fit_rule` **in** `s06_ks.py`; **2,499 quarters used**.

© Zhigang Feng Redistribution without permission

3 · Push a Histogram, Not a Panel — and Never Let a Grid Clamp

Two choices that make the regression trustworthy: no sampling error in the simulated K_t, and no capital outside the grid that prices it.

A histogram. Krusell and Smith simulated a panel of households; this code pushes the distribution over (wealth, employment) on 3,001 nodes 0.1 apart with 6.A2's lottery, as the simulate frame showed. The series K_t then carries no sampling error on either side of the regression (6.A2 measured what a panel costs). Employment shares then match the published unemployment rates exactly (error 1.5×10^{-14}).

Never clamp in silence

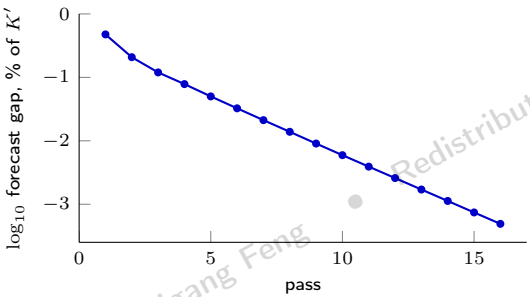
Simulated K and every forecast K' must stay strictly inside the K-grid $[30, 50]$. If not, the code stops, widens the grid and restarts. Here K stayed in $[37.08,\ 42.08]$, and the grid was never widened.

A clamp would read a policy at the wrong K and report a tidy rule anyway: this block's version of the careless bracket of 6.A2.

© Zhigang Feng • Redistribution prohibited without permission

Ledger `ks.gates`, `ks.solution` (`hist_nodes`, `K_grid_widened`).

Sixteen passes. Judge them by the forecasts, not the coefficients: intercept and slope move against each other.



Measured

16 passes, 7,360 household sweeps in all, 13 s. The forecast gap between the rule used and the rule implied fell from 0.475% of K' to 0.0005%; the coefficients moved 7×10^{-6} at the last pass.

From a stored cross-section and rule, three passes take 2 s: Lab 6 runs those.

Ledger `ks.solution.rows`, `fig_ks_convergence`, `ks.short_run`.

© Zhigang Feng · Redistribution prohibited without permission

Two Ways to Grade the Rule: One Step, or the Whole Path

Both compare the rule's forecast with the capital the simulated economy actually produced. They differ in what the rule is allowed to see.

The rule. $\log \hat{K}_{t+1} = \kappa_0(z_t) + \kappa_1(z_t) \log K_t$, with $\kappa_1 \approx 0.965$.

One step (R^2, the regression's own score). Every quarter, feed the rule the *true* K_t and predict K_{t+1}. Next quarter, start again from the true K_{t+1}. A mistake never carries over.

Den Haan's test. Feed the rule the true K_0 *once*. After that it sees only the shocks z_t and its own forecasts:

$$\hat{K}_{t+1} = H(\hat{K}_t, z_t), \qquad e_t = 100 \left| \log \hat{K}_t - \log K_t \right|.$$

This is how a household uses the rule: it forecasts many quarters ahead from today.

Why the two numbers differ

On the whole path, today's error is passed on with weight κ_1, and the new one-step error is added: $e_{t+1} \approx \kappa_1 e_t +$(one-step error). Errors can pile up to $1/(1-\kappa_1) \approx 28$ times the one-step size.

Pitfall

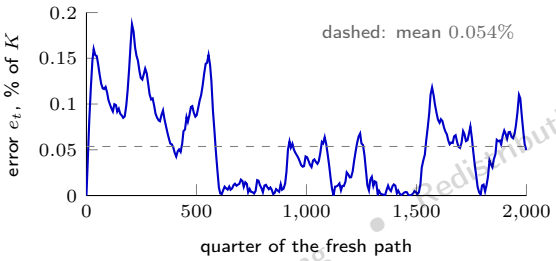
K moves slowly, so the true K_t alone predicts K_{t+1} well. R^2 is close to 1 for almost any rule and cannot tell a good rule from a poor one.

© Zhigang Feng · Redistribution prohibited without permission

Den Haan, *JEDC* 34 (2010) — the test 2.B1 named. `fit_rule`, `rule_path` in `s06_ks.py`; ledger `ks.solution` (κ).

4 · Den Haan's Test, Run on a Fresh Path

What it measures: the rule as a many-quarter forecaster — started at the true K, then fed only the shocks and its own forecasts — against the K_t the economy produced, on 2,000 fresh quarters.



Measured, same rule

	mean	max
one step	0.002%	0.013%
Den Haan	0.054%	0.187%

Reading the figure

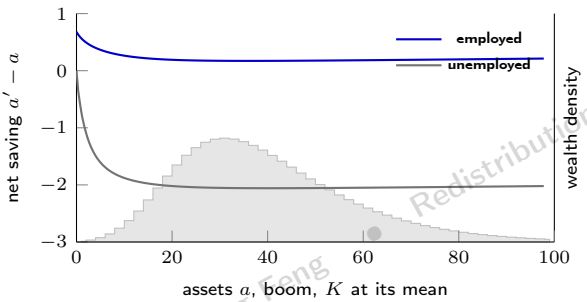
$e_t = 100\,|\log \hat{K}_t - \log K_t|$: zero at $t = 0$, grows while one-step errors pile up, shrinks when the shocks turn, never trends. Mean $24\times$ the one-step error, under 0.2% of K: the rule passes.

Fresh: 2,000 shocks not used in the fit. **On the fitting path (2,500 quarters)**: **mean** 0.054%, **max** 0.150%. **Ledger** `ks.diagnostics`, `ks.fig_den_haan_error`.

© Zhigang Feng — Redistribution prohibited without permission

Why One Moment Is Nearly Enough

Where the wealth is, saving is a straight line in wealth with one slope. Moving wealth between such households does not change total saving.



© Zhigang Feng
Redistribution prohibited without permission

Measured

A straight line fits $a'(a)$ with mass-weighted $R^2 = 0.9999985$ for the employed and 0.999993 for the unemployed; the slopes are 0.9998 and 0.9991. Only 0.003% of households sit at the limit, where the line bends.

A property of a calibration

Not a theorem (2.B1). Fat right tails, many households at the limit, or a policy that moves them would break it.

Ledger `ks.fig_ks_policy` (`linear_fit`), `ks.diagnostics.wealth`; **grey: the time-averaged wealth distribution**.

5 · Where Moments Run Out — and What the Whole Distribution Would Cost

Adding moments buys accuracy and costs a dimension each. And a government choosing policy needs what no moment gives: how the whole distribution responds.

Three problems the mean cannot fix:

- no guidance on which moments, or what functional form;
- welfare and inequality need the shape, not two moments;
- a government searching over policies needs $\partial V/\partial \Gamma$ at every policy it might choose — aggregation was verified at one.

Keep Γ itself, then. 6.A's grid holds it in $1{,}400$ numbers. A Smolyak grid over them (5.B3): $2{,}801$ points at $\mu = 1$, $3{,}922{,}801$ at $\mu = 2$.

Count the work

Each point needs a household and a distribution: 0.12 s in 6.A. At $\mu = 2$ that is 5.3 days — for *one* evaluation of a government's objective, inside a search over policies.

Not only arithmetic

A distribution lives on a simplex, not a box; relabelling households should change nothing; simulation destroys derivatives (6.A2). 7.B4 and Session 8 take them up.

© Zhigang Feng · Redistribution prohibited without permission

Ledger `wall`: **Smolyak counts $2d+1$ and $2d^2+2d+1$, checked against every row of the Session 4 ledger**; `core.inner_solve_seconds`.

How It Works (1): VFI on (a, s, K)

```
def ks_expected_value(V, th, m0, P):
    """E[V(a', s', K') | s, K] on the asset nodes.  K' = forecast(z, K) lies between the
    K-nodes m0 and m0 + 1: read V there with weights 1 - th and th (linear in log K)."""
    Vn = (1.0 - th) * V[:, :, m0] + th * V[:, :, m0 + 1]        # V(a', s', K'(z, K)): (nA, 4, 2, nK)
    # Z_OF_S = [0, 0, 1, 1]: today's z for each s (s = 0, 1 in a recession; 2, 3 in a boom)
    return np.einsum("sp,apsm->asm", P, Vn[:, :, Z_OF_S, :])  # sum over s' with P[s, s']
```

```
    for it in range(1, max_iter + 1):
        EV, V_new = ks_expected_value(V, th, m0, P), np.empty_like(V)   # E V at K'(z, K)
        for s in range(4):                                              # (employment, z) today
            for m in range(nK):                                         # K-node today
                vhat = interpolant(interp, a, EV[:, s, m])      # V between asset nodes
                obj = lambda x: u(coh[:, s, m] - x, p.gamma) + p.beta * vhat(x)   # the objective
                ap[:, s, m], V_new[:, s, m], _ = golden_max(obj, p.a_min, top[:, s, m],
                                                            n_golden)   # a' by golden section
        history.append(float(np.abs(V_new - V).max()))
        V = V_new
        if history[-1] < tol:
            converged = True
            break
        for _ in range(n_howard):                                       # Howard: keep a', update V
            EV = ks_expected_value(V, th, m0, P)
            for s in range(4):
                for m in range(nK):
                    vhat = interpolant(interp, a, EV[:, s, m])
                    V[:, s, m] = (u(coh[:, s, m] - ap[:, s, m], p.gamma)
                                  + p.beta * vhat(ap[:, s, m]))
        if max_seconds is not None and time.perf_counter() - t0 > max_seconds:
            break
```

© Zhiguo Feng • Redistribution prohibited without permission

How It Works (2): The Inner Problem by the Euler Route (What the Loop Runs), and the Grid Guard

```
    for it in range(1, max_iter + 1):                          # one EGM sweep, kappa fixed
        cn = (1.0 - th) * c[:, :, m0] + th * c[:, :, m0 + 1]   # c'(a', s', K'): K' from the rule
        MU = Rp[None] * cn ** (-p.gamma)                       # R' u'(c')
        Euc = np.einsum("sp,apsm->asm", P, MU[:, :, Z_OF_S, :])  # E[R' u'(c') | s]
        c_end = (p.beta * Euc) ** (-1.0 / p.gamma)             # invert the Euler equation
        a_end = (c_end + a[:, None, None] - inc[None]) / R[None]   # endogenous grid
        c_new = _back_to_grid(a, a_end.reshape(nA, -1), c_end.reshape(nA, -1),
                              Rf, incf, p.a_min)               # limit: a comparison
        c_new = np.maximum(c_new.reshape(nA, 4, nK), 1e-10)
        diff = float(np.abs(c_new - c).max())
        c = c_new
        if diff < tol:
            break
```

```
def check_rule_inside(kappa, logK):
    """The forecast must map the K grid into itself, or the household would read a
    clamped policy: stop and say so."""
    logKp = kappa[:, 0][:, None] + kappa[:, 1][:, None] * logK[None, :]
    if logKp.min() <= logK[0] or logKp.max() >= logK[-1]:
        raise KGridError(f"forecast K' in [{np.exp(logKp.min()):.2f}, "
                         f"{np.exp(logKp.max()):.2f}] reaches the K grid "
                         f"[{np.exp(logK[0]):.2f}, {np.exp(logK[-1]):.2f}]: widen it")
    return logKp
```

Redistribution prohibited without permission
© Zhiming Cheng

How It Works (3): The Loop

```
for outer in range(1, s.max_outer + 1):
    t1 = time.time()
    try:                                                # 1. household  2. simulate
        c, it, _ = solve_household_ks(kappa, p, a, logK, P, c_init=c,
                                      tol=s.tol_egm, max_iter=s.max_egm)
        sim = simulate(c, p, a, logK, Pz, P, z, lam0, h)
    except KGridError as err:                           # never clamp: widen, restart
        widened += 1
        if widened > 3:
            raise
        lo, hi = np.exp(logK[0]) * 0.85, np.exp(logK[-1]) * 1.15
        logK, c = np.linspace(np.log(lo), np.log(hi), s.n_K), None
        _say(verbose, f"   {err}\n   K grid widened to [{lo:.1f}, {hi:.1f}]")
        continue
    sweeps += it
    kappa_hat, r2 = fit_rule(sim["K"], z, s.T_burn)  # 3. regress, state by state
    change = float(np.abs(kappa_hat - kappa).max())
    rows.append(_pass_record(outer, it, change, kappa, kappa_hat, sim, r2, t1,
                             verbose))
    if change < s.tol_coef:
        break
    kappa = (1.0 - s.damping) * kappa + s.damping * kappa_hat     # 4. damped update
    lam0 = restart_distribution(sim["lam_end"], p.u[z0])
```

© Yiqiang Feng · Redistribution prohibited without permission

How It Works (4): One Quarter of the Histogram, the Regression, the Test

```
        moved = np.empty((nH, 2))
        for e in range(2):                          # a' on the histogram nodes h
            coh = (1.0 - delta + r) * h + income[e]  # r, w, income all from (K_t, z_t)
            ap = np.clip(coh - np.interp(h, a, ct[:, e]), p.a_min, h[-1])   # policy read at K_t
            q = (ap - h[0]) / dh                    # Young's lottery, uniform grid
            j = np.minimum(q.astype(np.int64), nH - 2)   # the lower node ...
            w_hi = q - j                            # ... and the share sent up
            moved[:, e] = (np.bincount(j, weights=lam[:, e] * (1.0 - w_hi), minlength=nH)
                           + np.bincount(j + 1, weights=lam[:, e] * w_hi, minlength=nH))
        zj = z[t + 1]                               # employment moves, given z -> z'
        lam = moved @ (P[s0:s0 + 2, 2 * zj:2 * zj + 2] / Pz[zi, zj])   # lambda_{t+1}
```

```
def fit_rule(K, z, start):
    """Least squares of log K' on log K, separately by today's aggregate state."""
    x, y, zz = np.log(K[start:-1]), np.log(K[start + 1:]), z[start:-1]   # after the burn-in
    kappa, r2 = np.empty((2, 2)), np.empty(2)
    for zi in range(2):                                  # recession, boom
        xi, yi = x[zz == zi], y[zz == zi]                # the quarters with z_t = zi
        A = np.column_stack([np.ones_like(xi), xi])      # regressors: 1, log K_t
        b, *_ = np.linalg.lstsq(A, yi, rcond=None)       # OLS: (kappa_0, kappa_1)
        kappa[zi] = b
        r2[zi] = 1.0 - ((yi - A @ b) ** 2).sum() / ((yi - yi.mean()) ** 2).sum()  # R^2
    return kappa, r2
```

```
def rule_path(kappa, K_start, z):
    """Iterate the perceived law forward from one K, never looking at the simulation."""
    Kf = np.empty(z.size)
    Kf[0] = K_start                                      # start at the true K, once
    for t in range(z.size - 1):                          # then forecast from the forecast
        Kf[t + 1] = np.exp(kappa[z[t], 0] + kappa[z[t], 1] * np.log(Kf[t]))   # z_t's row of kappa
    return Kf
```

© Zhigang Feng · Distribution prohibited without permission

Takeaways

- **Part 1.** With an aggregate shock the distribution is a state. Krusell and Smith forecast it by its mean: $\log K' = \kappa_0(z) + \kappa_1(z) \log K$, a fixed point in four numbers.
- **Part 2.** The inner problem: the household with four states, κ fixed. VFI reads V at the forecast K' (linear in $\log K$, then in a): 1,382 sweeps plain, 62 maximizations with Howard. EGM: 0.31 s. They agree to 0.1% where households are.
- **Part 3.** The outer problem: solve the inner one, push a histogram 3,000 quarters for the series K_t, regress $\log K_{t+1}$ on $\log K_t$ once per aggregate state, damp. No panel, no silent clamp.
- **Part 4.** 16 passes. $R^2 = 0.999999$ grades one-step forecasts from the true K_t and says little; Den Haan's error grades the rule run on its own forecasts: 0.19% at worst on a fresh path, so it passes.
- **Part 5.** One moment works because saving is linear in wealth where the wealth is. A government needs the whole distribution: 3.9 million Smolyak points at $\mu = 2$.

© Zhigang Feng · Redistribution prohibited without permission

Bridge: One Surprise Instead of a Shock That Keeps Coming

Krusell–Smith's distribution moves because the aggregate shock never stops. Remove the shock, and change one thing once: a tax, announced today, for ever.

After that surprise nothing is random in the aggregate. Prices become a *sequence* rather than a function of the distribution — and the problem becomes a path to compute.

Next: 6.B2

An MIT shock: first in the representative-agent growth model, solved three ways; then the same shock in Block A's economy, with a distribution; then who gains. And the life cycle, where cohorts are the distribution.

© Zhigang Feng · Reproduction prohibited without permission